

Deep Learning Architectures for Multilingual Code-Mixed Indian Language Summarization: A One-Month Deployment Framework

The proliferation of digital communication platforms has fundamentally altered the linguistic landscape, particularly in highly multilingual societies such as India. The resulting digital discourse is heavily characterized by intense code-mixing—the fluid, dynamic alternation of two or more languages within a single utterance or document. Consequently, traditional natural language processing systems, which inherently operate under monolingual assumptions and rigid orthographic constraints, experience significant performance degradation when confronted with code-mixed data. Building an automated, abstractive text summarization system capable of parsing, understanding, and condensing code-mixed Indian languages, specifically language pairs such as Hindi-English (Hinglish), Tamil-English (Tanglish), and Bengali-English (Banglish), requires navigating complex orthographic variations, shifting grammatical frameworks, and severe data scarcity.

Deploying such a sophisticated deep learning architecture within a strict one-month timeframe demands a highly optimized, pragmatically constrained, yet academically rigorous approach. The engineering constraints explicitly prohibit the use of external translation APIs, mandating a localized, model-driven solution capable of handling multilingual inputs natively. This report delineates a comprehensive, expert-level blueprint for engineering an end-to-end transformer-based summarization pipeline. The proposed strategy systematically addresses these constraints by leveraging region-specific sequence-to-sequence modeling, algorithmic synthetic data generation based on linguistic theories, and a multi-phase curriculum learning strategy to achieve robust generalization across highly erratic code-mixed inputs.

The Linguistic and Computational Context of Code-Mixing

To effectively engineer a system capable of summarizing code-mixed text, it is imperative to first deconstruct the linguistic phenomena at play. Code-mixing is not merely the random insertion of foreign words into a sentence; it is a highly structured, rule-governed communicative behavior. Linguistically, it manifests in three primary forms: inter-sentential mixing (alternating languages across sentence boundaries), intra-sentential mixing (alternating languages within a single sentence clause), and tag switching (inserting a tag or phrase from one language into an utterance entirely in another).¹ In the context of Indian social media and informal digital communication, intra-sentential mixing is the most prevalent and the most

computationally challenging.³

The computational challenge is exacerbated by orthographic diversity. Indian languages span multiple distinct script families, including Indo-Aryan (Devanagari, Bengali, Gujarati) and Dravidian (Tamil, Telugu, Kannada) scripts.⁴ However, user-generated code-mixed text frequently employs the Latin (Roman) alphabet to represent native phonetic sounds—a phenomenon known as informal romanization.⁵ This introduces severe spelling inconsistencies, phonetic ambiguities, and a high degree of out-of-vocabulary tokens. For example, the Hindi phrase for "How are you?" might be transcribed as "aap kaise ho," "ap kaisey ho," or "aap kase ho," all representing the same semantic meaning but mapping to entirely different token representations in a naive natural language processing pipeline.⁶

Furthermore, the morphological structures of the languages involved often collide. Dravidian languages like Tamil are highly agglutinative, meaning they build complex words by attaching multiple suffixes to a root stem. In Tanglish, it is common to observe English root words conjugated with Tamil morphosyntactic suffixes (e.g., "wait-pannu" for "please wait"). A deep learning model must be capable of disentangling these root-suffix amalgamations without relying on explicit, handcrafted grammatical rules.⁸ The goal of this report is to construct a transformer-based architecture that can seamlessly ingest these noisy, non-standardized inputs and produce coherent, accurately summarized outputs within a tightly constrained development window.

Architectural Approach Selection

When engineering a sequence-to-sequence summarization model for code-mixed languages under tight temporal constraints, the architectural approach dictates the trajectory, feasibility, and ultimate success of the entire deployment. Three primary strategies dominate the current computational linguistics literature regarding multilingual and code-mixed modeling.

The first strategy is the pretraining of a language model from scratch on a massive corpus of code-mixed text, followed by task-specific adaptation. While this approach yields models deeply attuned to the nuances of code-mixing, it is computationally prohibitive and entirely unfeasible within a one-month development cycle. Training a transformer model from scratch requires terabytes of text, distributed computing clusters containing hundreds of highly specialized graphics processing units, and months of uninterrupted training time.⁹ Furthermore, authentic, large-scale, unannotated code-mixed corpora are notoriously scarce, rendering this approach practically impossible for rapid prototyping.

The second strategy involves the direct fine-tuning of existing, massively multilingual pre-trained transformer models, such as the Multilingual Text-to-Text Transfer Transformer (mT5), the Multilingual Bidirectional and Auto-Regressive Transformers (mBART), or IndicBART, directly on downstream summarization tasks utilizing available datasets.¹⁰ This approach leverages the cross-lingual transfer capabilities inherent in these models, which have been exposed to vast amounts of monolingual data across dozens of languages during their initial

pretraining phases. While computationally efficient and easily executed within a one-month timeframe, direct adaptation on standard monolingual summarization datasets (such as the Indian Language Summarization dataset, ILSUM) often fails to capture the intricate syntax of code-mixed text. When evaluated on code-mixed inputs, these models frequently suffer from target-language confusion, hallucinating summaries in the wrong language or failing to comprehend the structural shifts of intra-sentential mixing.¹²

The third strategy focuses on augmenting the training pipeline through the generation of synthetic code-mixed datasets. Because high-quality, human-annotated code-mixed summarization pairs are exceedingly rare, synthetic data generation serves to bridge the critical resource gap. By algorithmically introducing code-mixing phenomena into rich, high-quality monolingual datasets, practitioners can artificially construct parallel corpora that teach the model cross-lingual alignment and code-switched syntactic structures without requiring massive computational pretraining budgets.¹⁵

The optimal architectural decision for a one-month deployment is a hybrid framework: **Fine-Tuning a Region-Specific Transformer utilizing Synthetically Augmented Data and Curriculum Learning**. Attempting to pretrain a model from scratch is impossible, and relying solely on scarce authentic code-mixed datasets will yield suboptimal, brittle results. Instead, the most pragmatic and technically rigorous pathway involves selecting a pre-trained sequence-to-sequence model that is already specialized for the Indian linguistic subcontinent. This model is then fine-tuned using a heavily augmented dataset, where the high-quality monolingual data from the ILSUM corpus is synthetically transformed into code-mixed data. This synthetic data, carefully combined with the limited available authentic code-mixed corpora, forms the foundation of a staged curriculum learning training regimen.¹⁷ This hybrid approach entirely circumvents the need for external translation application programming interfaces, rigidly respects computational and temporal limits, and aggressively targets the distinct nuances of mixed-language syntax.

Foundation Model Selection: The IndicBART Imperative

Selecting the correct foundational transformer architecture is the most consequential decision in the entire engineering pipeline. The chosen model must possess an inherent capacity for multilingual generation, a vocabulary structure capable of handling diverse Indic scripts, and a parameter footprint small enough to allow for rapid, iterative fine-tuning on consumer-grade or standard academic hardware within a four-week window.

While globally recognized models like mT5 and mBART50 offer expansive multilingual support and boast state-of-the-art performance on numerous benchmarks, they present significant drawbacks for this specific, constrained deployment scenario. The Multilingual T5 (mT5) model, for instance, is highly capable and was utilized successfully by several top-performing teams in the ILSUM 2024 shared tasks.⁴ However, its massive vocabulary size, which is designed to support over a hundred languages globally, means that its parameter space is spread

exceptionally thin. It is not optimized specifically for the linguistic structures or the specific vocabulary distributions of the Indian subcontinent.¹¹ Similarly, mBART50 is a powerful denoising autoencoder designed for sequence-to-sequence generation tasks, but its 610 million parameters require significant computational overhead, complicating rapid experimentation, hyperparameter tuning, and memory management during fine-tuning.⁹

The definitively superior recommendation for this project is **IndicBART**. Developed explicitly for natural language generation tasks across Indian languages, IndicBART offers several critical architectural advantages that make it uniquely suited for code-mixed Indic summarization.⁹

The most profound advantage of IndicBART is its implementation of orthographic unification. Many Indic languages utilize scripts derived from the ancient Brahmi script system. Recognizing this inherent connection, IndicBART leverages orthographic similarity by mapping all Indic language data to a single script—Devanagari—internally before processing.⁹ This script unification is a masterstroke in model efficiency; it reduces the number of distinct scripts processed from multiple down to one, thereby massively increasing the shared subwords in the model's vocabulary. Because subword embeddings typically consume a significant fraction of a transformer's parameter space, this single-script modeling approach allows IndicBART to dedicate substantially more computational capacity and parameter weight to higher-level semantic learning and cross-lingual alignment rather than redundant character mapping across visually distinct but phonetically identical scripts.⁹

Furthermore, IndicBART boasts exceptional parameter efficiency. At 244 million parameters, it is significantly more compact than mT5-base (580 million parameters) or mBART50 (610 million parameters).¹⁹ This smaller footprint is a critical asset for a student or researcher operating under a one-month deadline. It enables faster training epochs, permits larger batch sizes on standard graphics processing units (such as the NVIDIA T4 or V100 commonly available on cloud platforms), and facilitates rapid, iterative debugging cycles.

Finally, IndicBART benefits from domain-specific pretraining. The model was pre-trained on the massive IndicCorp dataset, which contains high-quality news crawls for a wide array of Indian languages, as well as Indian English.⁹ Because the primary summarization dataset available for this project, ILSUM, is also heavily grounded in news articles and headlines²⁰, the domain overlap is nearly perfect. This ensures that IndicBART inherently understands the stylistic, rhetorical, and topical distributions of the target data, requiring significantly less fine-tuning to achieve coherent abstractive summarization. Empirical evaluations from the ILSUM shared tasks and independent natural language generation benchmarks continuously demonstrate that IndicBART consistently outperforms or remains highly competitive with larger universal models like mBART50 on Indic language summarization, particularly in low-resource and code-mixed settings.⁹

Foundation Model	Parameter Count	Script Handling	Suitability for 1-Month Code-Mixed
------------------	-----------------	-----------------	------------------------------------

		Architecture	Project
mT5-base	580 Million	Distinct encoding for over 100 global scripts, massive vocabulary overhead.	Low to Medium. Computationally heavy, generalized vocabulary dilutes Indic performance.
mBART50	610 Million	Distinct encoding for 50 languages.	Low. High memory footprint restricts batch sizing and slows iteration speed.
MuRIL	236 Million	Optimized for Indic languages, but fundamentally an encoder-only architecture.	Low. Excellent for classification, but requires a complex bolted-on decoder for summarization.
IndicBART	244 Million	Orthographic unification to Devanagari script for 11 Indic languages.	High (Recommended). Computationally efficient, perfectly aligned domain pretraining, superior script handling.

Engineering the Data Ecosystem: ILSUM, GupShup, and Beyond

The fundamental axiom of deep learning is that model performance is inextricably linked to the quality, volume, and relevance of the training data. For code-mixed Indian language summarization, constructing the appropriate data ecosystem is a formidable challenge. While high-quality datasets exist, they are fragmented across different tasks and formats. A

successful one-month deployment relies on intelligently amalgamating these disparate sources into a cohesive training corpus.

The cornerstone of the project's data strategy is the **Indian Language Summarization (ILSUM) Dataset**. Developed specifically to bridge the resource gap for Indian language natural language processing, the ILSUM dataset provides approximately 15,000 to 20,000 high-quality, monolingual news article-summary pairs per language.²⁰ The dataset covers major languages including Hindi, Gujarati, Bengali, and Indian English, with recent editions expanding to include Dravidian languages such as Kannada, Tamil, and Telugu.²³ The summaries within ILSUM are a mix of extractive and abstractive generation, utilizing human-written news headlines as the ground truth summaries.⁴ While ILSUM provides the structural foundation for teaching the model the fundamental mechanics of text summarization, its limitation lies in its monolingual nature. Although the creators note the presence of incidental code-mixing and script-mixing within the articles (e.g., English phrases borrowed into Hindi text), it lacks the dense, highly varied intra-sentential code-mixing typical of colloquial digital communication.⁴

To introduce the model to authentic code-mixing behavior, the pipeline must integrate specialized conversational datasets. The most critical resource in this category is the **GupShup Corpus**. GupShup is a meticulously curated, code-switched conversational summarization corpus containing 6,831 open-domain Hindi-English conversations. Crucially, each conversation is paired with human-annotated abstractive summaries in both pure English and Hindi-English code-switched text.²⁴ This dataset is invaluable as it provides the exact input-output mapping required for the final stages of model fine-tuning, demonstrating how humans naturally summarize chaotic, mixed-language dialogue into concise, code-mixed representations.²⁶

To ensure the model generalizes across multiple language pairs, particularly the requested Tamil-English dynamic, the pipeline should incorporate supplementary datasets designed for other natural language understanding tasks. The **DravidianLangTech** shared tasks provide extensive Tamil-English, Malayalam-English, and Kannada-English code-mixed datasets, originally curated for sentiment analysis and offensive language identification from YouTube comments.² While these datasets lack summarization labels, the input text is rich in authentic Tenglish code-mixing, encompassing inter-sentential, intra-sentential, and tag-switching behaviors.² These datasets can be leveraged during intermediate pretraining or utilized within a multi-task learning framework to expose the model's encoder to the structural realities of Dravidian code-mixing before fine-tuning the decoder for summarization. Similarly, the **HinGE** (Hinglish Generation and Evaluation) dataset provides human-generated Hinglish sentences corresponding to parallel Hindi-English text, offering another rich source of authentic syntactic structures.²⁹

Algorithmic Synthesis of Code-Mixed Data

Despite the utility of the GupShup dataset, 6,831 examples are insufficient to train a deep learning model to generalize effectively across the vast permutations of code-mixing, especially when expanding beyond Hindi to languages like Tamil. Because external translation

APIs like Google Translate are strictly prohibited by the project constraints, the strategy mandates the algorithmic synthesis of code-mixed summarization data directly from the large monolingual ILSUM dataset.¹⁵ This local synthesis is achieved through a bipartite algorithmic approach.

The first approach is **Bilingual Lexicon Replacement**, a highly efficient heuristic method for generating intra-sentential code-mixing.³¹ This technique mimics the real-world linguistic phenomenon of lexical borrowing, where bilingual speakers utilize the grammatical framework of their native language but substitute specific content words (usually nouns, adjectives, or adverbs) with English equivalents. To implement this, the pipeline utilizes open-source bilingual dictionaries, such as the Multilingual Unsupervised and Supervised Embeddings (MUSE) dictionaries published by Facebook Research, or the extensive PanLex database.³² The monolingual Hindi or Tamil text from the ILSUM dataset is processed using a fast Part-of-Speech (POS) tagger via the Natural Language Toolkit (NLTK) or the Stanford NLP library.³⁵ Specific target syntactic components are identified and systematically swapped with their cross-lingual counterparts from the bilingual dictionary based on a defined probability distribution.³¹ For instance, a pure Hindi sentence regarding the economy would have its nouns computationally swapped with English translations, generating a synthetic Hinglish sentence that maintains Hindi syntax but utilizes English vocabulary. This process requires minimal computational overhead and generates vast quantities of lexically mixed data.

The second, more advanced approach utilizes **Linguistic Theory-Based Generation** to create complex, structurally mixed data. The pipeline leverages the open-source Generate Code-Mixed (GCM) toolkit developed by Microsoft Research.³⁸ The GCM toolkit operates entirely offline and requires only parallel monolingual corpora as input. It utilizes linguistic alignment tools like `fast_align` to map word-level relationships between two languages.³⁸ Crucially, the GCM toolkit implements two prominent linguistic theories of code-mixing to govern the generation process: the Matrix Language Theory and the Equivalence Constraint Theory.¹⁵ The Matrix Language Theory posits that in a code-mixed utterance, one dominant language (the matrix language) dictates the overarching morphosyntactic frame, while the embedded language provides specific content morphemes. The Equivalence Constraint Theory dictates that code-switching can only occur at points where the surface grammatical structures of both languages map onto each other seamlessly, preventing ungrammatical word orders. By applying these rigid theoretical constraints to parallel sentences, the GCM tool generates all grammatically valid permutations of code-mixed text for a given sentence pair.³⁹ A sophisticated sampling algorithm then selects the most natural-sounding variants, providing a massive, highly coherent synthetic dataset that teaches the transformer model the deep structural rules of code-switching.¹⁵

The Preprocessing and Normalization Pipeline

The architecture of the transformer model is ultimately only as effective as the quality of the data it ingests. Processing code-mixed inputs requires an exceptionally robust, error-tolerant

preprocessing pipeline that cleans, standardizes, and unifies noisy text before it reaches the tokenizer's subword splitting mechanisms. Code-mixed data extracted from social media, news comments, or conversational sources is highly irregular. It suffers from phonetic spelling variations, a complete lack of grammatical standardization, and the chaotic blending of distinct writing systems.⁵ The end-to-end pipeline must systematically dismantle these complexities through a sequence of deterministic transformations.

1. Orthographic Normalization and Cleanup

The raw input text often contains a chaotic mixture of invisible formatting characters, zero-width joiners (ZWJ), inconsistent punctuation, and non-standard whitespace. In Indian languages, the representation of complex characters can be highly varied; a single visual character might be encoded via multiple different Unicode sequences depending on the input device used by the author.

The first step in the preprocessing pipeline utilizes the IndicNLP library, specifically the IndicNormalizerFactory module.⁴¹ This component standardizes the input by removing invisible formatting characters, standardizing the representations of Nukta-based characters (dots added to consonant letters to represent foreign sounds), and resolving multiple encodings of two-part dependent vowel signs.⁴¹ Normalization is non-negotiable in this architecture. Failing to normalize results in the Byte-Pair Encoding tokenizer treating visually identical words as separate, distinct entities, thereby artificially inflating the vocabulary space, diluting embedding representations, and severely degrading model performance.⁴³

2. Transliteration and Script Unification

The most severe challenge in processing code-mixed Indian text is the pervasive use of informal romanization—writing native languages using the Latin alphabet. Recent empirical studies on the reliability of large language models in healthcare settings revealed that models experience a massive 5-to-12 point drop in F1 performance when processing Romanized Indic text compared to text in native scripts.⁴⁴ If a user inputs the phrase *"Aaj ka weather kaisa hai?"*, the model must recognize *"Aaj"* and *"kaisa"* as Hindi tokens, despite their Latin characters. Because the chosen model, IndicBART, operates optimally when scripts are unified to Devanagari⁹, the pipeline must actively transliterate Romanized Indic words into their native orthography prior to tokenization.⁴⁵

To achieve this without relying on external APIs, the pipeline integrates the indic-transliteration Python package.⁴⁶ This library supports highly accurate Roman-to-Native back-transliteration using predefined mapping schemes.⁴⁷ By passing the input stream through a language identification heuristic, English words are preserved in the Latin script, while Romanized Hindi or Tamil words are intercepted and transliterated back into Devanagari or the Tamil script, respectively. This forces the noisy input to conform to the optimized vocabulary space of the IndicBART model.⁴⁸

3. Phonetic Spelling Standardization

Informal code-mixing is characterized by severe spelling inconsistencies. A single word like "aapke" (yours) might be spelled as "apke," "apake," or "aapkey" depending on the user's phonetic interpretation.⁴⁹ These variations drastically increase the out-of-vocabulary rate and confuse the model's contextual understanding.⁶ To mitigate this, the pipeline implements phonetic matching algorithms. Utilizing the pyphonetics Python library, which includes implementations of the Soundex and Metaphone algorithms, the preprocessing script converts words into their phonetic representations.⁴⁹ Words that share the same phonetic hash within a defined edit distance are clustered and replaced with a single, standardized spelling variant based on a curated dictionary. This computationally inexpensive technique drastically reduces vocabulary sparsity and improves the correlation between syntactic and semantic arrangements.⁶

4. Tokenization and Sequence Framing

Once the text is normalized, transliterated, and phonetically standardized, it is processed by the tokenizer. IndicBART utilizes an ALBERT-based or mBART-based tokenizer trained via Byte-Pair Encoding (BPE).⁵⁰ BPE is highly advantageous for code-mixed text because it breaks down unknown slang or mixed-morphology words into known subword units.⁵¹ For example, in Tamil-English mixing, the word "drive-pannu" (to drive) would be intelligently split by the BPE tokenizer into the English root "drive" and the Tamil suffix subwords, allowing the model to process the distinct linguistic components independently.

The tokenized input sequence must then be framed according to the specific control tokens required by the IndicBART architecture. The input format strictly follows the structure: Sentence `</s>` `<2xx>`, where `</s>` is the end-of-sequence token and `<2xx>` is the explicit language identification tag (e.g., `<2hi>` for Hindi, `<2ta>` for Tamil, or `<2en>` for English).⁵⁰ In the context of code-mixed text, appending the dominant language tag of the sentence provides a vital conditioning signal to the decoder, explicitly guiding the language generation process and preventing target language hallucination.⁵²

Curriculum Learning and Fine-Tuning Strategy

Training a deep learning model directly on a chaotic amalgamation of formal monolingual news articles and highly erratic, colloquial code-mixed conversational data often causes severe gradient instability, leading to suboptimal convergence and catastrophic forgetting. To maximize performance and ensure robust generalization, the training regimen must implement a **Curriculum Learning** strategy.¹⁸

Curriculum learning involves scheduling the training data in a deterministic sequence of increasing complexity, mirroring the progressive nature of human language acquisition.¹⁷ By initially exposing the model to simple, standardized structures and gradually introducing complex, noisy data, the model avoids local minima and achieves significantly higher evaluation

scores. The training regimen for this one-month deployment spans four distinct phases.

Training Phase	Data Composition	Pedagogical Objective & Rationale
Phase 1: Monolingual Foundations	Pure ILSUM Dataset (Hindi, Tamil, English, etc.). ²³	Establish a strong foundational understanding of formal syntax, vocabulary, and the core structural mechanics of document summarization. ¹²
Phase 2: Lexical Code-Mixing	ILSUM data augmented via MUSE/PanLex dictionary noun/adjective swapping. ³¹	Introduce low-complexity intra-sentential mixing. The model learns to associate foreign vocabulary within a known, stable syntactic structure.
Phase 3: Structural Code-Mixing	Synthetic data generated via the GCM toolkit applying Equivalence Constraints. ³⁸	Expose the model to complex, phrase-level syntactic blending and varied grammatical structures, breaking the reliance on monolingual sentence frames.
Phase 4: Authentic Domain Adaptation	GupShup Code-Switched Conversational Summaries. ²⁵	Fine-tune the model on human-annotated, real-world code-mixing behavior, adapting the learned representations to informal conversational flow and slang. ²⁶

Hyperparameter Specifications for Fine-Tuning

The fine-tuning of the IndicBART model requires precise hyperparameter configuration. Because the model has already learned deep linguistic representations during its pretraining phase, aggressive hyperparameter settings risk catastrophic forgetting, where the model overwrites its foundational knowledge.

- **Optimizer:** The AdamW optimizer is standard for transformer architectures, providing robust weight decay to prevent overfitting on the relatively small code-mixed datasets.
- **Learning Rate:** A highly conservative learning rate is absolutely critical. Research indicates that aggressive learning rates during cross-lingual fine-tuning exacerbate the issue of the model generating text in the incorrect language. A peak learning rate of 5×10^{-5} is strongly recommended.¹⁴
- **Scheduling:** Implement a linear learning rate decay schedule, complemented by a warmup phase comprising 10% of the total training steps for each curriculum phase. This allows the optimizer to safely navigate the loss landscape before taking larger steps.
- **Batch Size & Gradient Accumulation:** Due to hardware constraints typical of a student project (e.g., a single GPU with 16GB VRAM), processing long summarization sequences will quickly cause out-of-memory errors. To counter this, a micro-batch size of 2 or 4 should be used, paired with gradient accumulation steps to simulate an effective batch size of 16 or 32. This provides the stable gradient estimates necessary for optimal convergence without exceeding memory limits.
- **Epochs:** Training should persist for 3 to 5 epochs per curriculum phase.⁵⁴ Extensive training beyond this point on synthetic data risks overfitting to the algorithmic artifacts generated by the synthesis tools.
- **Mixed Precision Training:** Implementing Automatic Mixed Precision (AMP), which stores specific activations and gradients in FP16 format rather than FP32, is highly recommended. AMP nearly halves VRAM requirements and accelerates computation speed by up to 40% on modern Tensor Core architectures, without a discernible loss in summarization accuracy.⁵⁵

Evaluation Metrics for Code-Mixed Generation

Evaluating the performance of natural language generation systems is inherently difficult, but evaluating code-mixed generation is notoriously complex. Standard evaluation metrics like basic ROUGE-N (Recall-Oriented Understudy for Gisting Evaluation) or BLEU are fundamentally inadequate for code-mixed text. These metrics rely on exact, rigid n-gram matching. Consequently, they heavily penalize synonymous transliteration variants (e.g., scoring "bhai" and "bhay" as a zero match), resulting in artificially low scores that do not reflect human judgments of summary quality.⁶ A robust evaluation framework for this project must move beyond simple surface-level matching.

The evaluation strategy must incorporate three advanced metrics to provide a holistic view of model performance:

1. **ROUGE-Lsum:** While standard ROUGE-N calculates rigid n-gram overlaps, ROUGE-Lsum calculates the Longest Common Subsequence (LCS) across the entire document-summary pair.⁵⁶ This approach accounts for sentence-level structure and flexible phrasing, making it highly suitable for evaluating the fluency, coherence, and structural integrity of abstractive summaries, even when the exact vocabulary differs

slightly from the reference.⁵⁶

2. **PhoBLEU**: To address the severe orthographic variations inherent in Hinglish and Tanglish, the pipeline must utilize PhoBLEU. This is a novel variant of the BLEU metric specifically engineered for code-mixed text. PhoBLEU adapts the metric phonetically, mapping generated words to phonetic representations before calculating overlap. This ensures that orthographic and spelling variations do not disproportionately degrade the evaluation score. Phonetic adaptation has been shown to align significantly better with human linguistic judgment (+0.10 to 0.15 on Pearson correlation) in code-mixed scenarios.⁵⁸
3. **BERTScore**: To measure true semantic fidelity rather than mere lexical overlap, the evaluation suite must include BERTScore. This metric utilizes pre-trained contextual embeddings to compute the cosine similarity between the generated summary and the reference summary at the token level.⁴ It successfully captures paraphrasing and synonymous substitutions that traditional metrics miss. During the ILSUM 2024 shared tasks, researchers reported a very strong positive correlation between BERTScore rankings and rigorous human evaluation for Indian language summarization, cementing its utility as a primary evaluation tool.⁴

Implementation Roadmap: The One-Month Deployment Timeline

Successfully executing an architecture of this magnitude within four weeks requires strict adherence to a modular, highly disciplined development timeline. Given the project constraint that approximately one week must be dedicated to learning the fundamentals of transformers, the roadmap is engineered to seamlessly integrate the educational phase with practical infrastructure setup.

Project Phase	Timeline Breakdown	Core Deliverables & Technical Milestones
Week 1: Theoretical Acquisition & Infrastructure Initialization	Days 1 - 7	Learning Phase: Focus heavily on the HuggingFace transformers ecosystem. Master the AutoModelForSeq2SeqLM, the Seq2SeqTrainer class, and BPE tokenization mechanics. Understand attention masks, padding, and the specific input

		framing required for encoder-decoder models. Infrastructure Phase: Initialize the Python environment. Download the massive ILSUM monolingual datasets and the GupShup code-mixed corpus.²⁰ Establish baseline evaluation scripts incorporating ROUGE-Lsum and BERTScore.⁵⁶
Week 2: Data Synthesis & Pipeline Construction	Days 8 - 14	Preprocessing Phase: Implement the text normalization scripts using IndicNLP to handle ZWJ removal and Nukta standardization.⁴¹ Implement the Roman-to-Native conversion scripts utilizing indic-transliteration.⁴⁶ Synthesis Phase: Build local bilingual lexicons using downloaded MUSE dictionaries.³² Execute the Microsoft GCM toolkit on a subset of the ILSUM parallel data to programmatically generate the Phase 3 structural code-mixed records.³⁸
Week 3: Curriculum Training Execution	Days 15 - 21	Training Phase: Instantiate the pre-trained IndicBART model (244M parameters) onto the available GPU

		hardware. ⁹ Configure the Seq2SeqTrainer with AdamW, learning rate schedules, and gradient accumulation. Execute Phase 1 (monolingual ILSUM) and Phase 2 (lexically swapped ILSUM) of the defined curriculum schedule. Monitor training loss carefully to prevent catastrophic forgetting.
Week 4: Advanced Tuning, Evaluation & Refinement	Days 22 - 28	Final Adaptation Phase: Execute Phase 3 (GCM synthetic data) and Phase 4 (GupShup authentic code-mixed data) fine-tuning.²⁵ Decoding Phase: Generate predictions on the hold-out test sets utilizing Beam Search (beam width 4-5) paired with rigorous length penalty configurations to ensure summary conciseness. Evaluation Phase: Calculate final ROUGE-Lsum, PhoBLEU, and BERTScore metrics. Document ablation studies and finalize the pipeline architecture for presentation and resume portfolio inclusion.

Baseline versus Advanced System Architectures

To ensure a functional, demonstrable deliverable at the end of the one-month period, the project development must be structured iteratively. Attempting to build the advanced,

multi-phase system simultaneously risks catastrophic project failure due to the compounding complexities of debugging NLP pipelines. Therefore, the implementation is bifurcated into a baseline system and an advanced system.

The **Baseline System**, targeted for completion by the end of Week 2, acts as a rapid-prototyping minimum viable product. It provides a fundamental empirical benchmark against which all subsequent algorithmic improvements will be measured. The baseline architecture utilizes a lightweight, easily accessible model, such as mT5-small, accessed via the HuggingFace API.⁵⁹ The data pipeline is exceedingly simple: naive, unaugmented fine-tuning directly on a concatenated mix of the ILSUM monolingual dataset and the GupShup dataset, with zero synthetic data generation or curriculum structuring.⁴ The input handling is similarly naive, feeding raw, unprocessed code-mixed strings directly into the tokenizer without transliteration or normalization. The expected outcome of this baseline is a model that successfully summarizes pure native scripts but exhibits severe degradation—including hallucinations, extreme repetition, and token fragmentation—when presented with Romanized slang or heavily code-mixed test samples.

The **Advanced System**, the ultimate goal for Week 4, integrates the deep technical nuances required for a resume-worthy, state-of-the-art computational linguistics deployment. The architecture shifts to IndicBART, leveraging its highly optimized unified Devanagari script mapping.⁹ The data pipeline is a full deployment of the Curriculum Learning schedule, progressing meticulously from ILSUM, to lexical swapping, to GCM-generated structural data, and concluding with the authentic GupShup corpus.¹⁸ Crucially, the input handling features the complete, robust preprocessing pipeline: normalization via IndicNLP, phonetic spelling standardization via pyphonetics, and forced transliteration of Latin characters to Native scripts via indic-transliteration prior to embedding lookup.⁴¹ The expected outcome is high structural cohesion, abstractive fluency, and semantic fidelity when summarizing heavily mixed Hinglish or Tamil-English text, accurately quantified by strong BERTScore and PhoBLEU evaluations.

Anticipated Challenges and Mitigation Strategies

Developing sequence-to-sequence generation systems for non-standardized linguistic structures introduces highly specific engineering risks. Preemptive, technically sound mitigation strategies are vital to ensure the project remains on schedule.

The foremost challenge is **Catastrophic Forgetting During Curriculum Learning**. As the model transitions from the formal, structured monolingual ILSUM data to the highly colloquial, noisy GupShup code-mixed data, it risks overwriting the fundamental summarization mechanics learned in Phase 1. To mitigate this, the training pipeline must implement a strategy known as experience replay. During the final fine-tuning phase on the GupShup dataset, the data loader should mix in a 10% to 15% random sample of the original monolingual ILSUM dataset. This constant trickle of formal data preserves the model's understanding of formal syntactic boundaries and prevents it from defaulting entirely to casual, unstructured generation styles.

A secondary challenge is the **Hallucination of Language Identification**. Multilingual generative models, particularly when processing heavily mixed input, can suffer from target-language confusion. For instance, the model might ingest a Tamil-English input and inexplicably generate a summary in Gujarati due to representational overlap in the vector space. To mitigate this, the pipeline must enforce strict, explicit conditioning via language tags. IndicBART utilizes specific syntax (e.g., <2en>, <2hi>, <2ta>).⁵⁰ The custom dataset mapping script must explicitly prepend these tags to both the encoder input sequence and the decoder target sequence. Furthermore, meticulous tuning of the learning rate prevents the cross-entropy loss gradients from overwhelming these subtle, yet vital, conditional signals during backpropagation.¹⁴

Finally, **Computational Bottlenecks and Out-Of-Memory (OOM) Errors** pose a constant threat. Even with the highly efficient 244 million parameter footprint of IndicBART, tracking millions of gradients across long sequence lengths (e.g., full 1000-word news articles) will result in instantaneous OOM errors on consumer-grade hardware. The primary mitigation strategy is the implementation of dynamic sequence truncation. The developer must analyze the token length distribution of the ILSUM dataset and truncate input tokens at the 90th percentile length (typically 512 or 768 tokens). This ensures that memory allocation remains predictable and within the physical limits of the hardware, allowing the project to proceed without the need for expensive cloud computing resources.

In conclusion, deploying a state-of-the-art deep learning model for code-mixed Indian language text summarization within a strict 30-day window is a highly demanding, yet eminently achievable engineering objective. By discarding computationally inefficient pretraining methods and adhering to the prohibition on external APIs, this proposed architecture focuses entirely on the strategic maximization of existing resources. The synthesis of the IndicBART model—with its profound advantage in unified script representation—coupled with a rigid, deterministic transliteration preprocessing pipeline effectively neuters the primary challenge of Romanized spelling variations. Furthermore, by programmatically generating massive volumes of synthetic code-mixed data using bilingual lexicons and linguistic constraint toolkits, the pipeline elegantly circumvents the critical scarcity of annotated mixed-language summarization pairs. Organizing this highly diverse data ecosystem through a phased Curriculum Learning strategy ensures stable gradient descent and highly robust semantic generalization, resulting in a resume-defining implementation of modern natural language generation.

Works cited

1. I am borrowing ya mixing ? An Analysis of English-Hindi Code Mixing in Facebook - ACL Anthology, accessed on April 3, 2026, <https://aclanthology.org/W14-3914.pdf>
2. Dravidian-CodeMix-FIRE 2021, accessed on April 3, 2026, <https://dravidian-codemix.github.io/2021/datasets.html>
3. Understanding Script-Mixing: A Case Study of Hindi-English Bilingual Twitter Users

- ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2020.calcs-1.5.pdf>
4. Overview of the Third Shared Task on Indian Language Summarization (ILSUM 2024) - CEUR-WS.org, accessed on April 3, 2026,
<https://ceur-ws.org/Vol-4054/T1-1.pdf>
 5. Improving Informally Romanized Language Identification - arXiv, accessed on April 3, 2026, <https://arxiv.org/html/2504.21540v1>
 6. Normalization of Spelling Variations in Code-Mixed Data - ACL Anthology, accessed on April 3, 2026, <https://aclanthology.org/2022.icon-main.33.pdf>
 7. Deep Learning Challenges for Code-Mixed Conversational Text | by Devanshi Bhatt | Spectrum Labs | Medium, accessed on April 3, 2026,
<https://medium.com/spectrum-labs/deep-learning-challenges-for-code-mixed-conversational-text-bcd6e48b9901>
 8. Advancing Sentiment Analysis in Tamil-English Code-Mixed Texts: Challenges and Transformer-Based Solutions - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2025.nlp4dh-1.27.pdf>
 9. IndicBART: A Pre-trained Model for Indic Natural Language Generation - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2022.findings-acl.145.pdf>
 10. Summarizing Indian Languages using Multilingual Transformers based Models - arXiv, accessed on April 3, 2026, <https://arxiv.org/abs/2303.16657>
 11. Automatic Data Retrieval for Cross Lingual Summarization - ACL Anthology, accessed on April 3, 2026, <https://aclanthology.org/2023.icon-1.85.pdf>
 12. PMIndiaSum: Multilingual and Cross-lingual Headline Summarization for Languages in India - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2023.findings-emnlp.777.pdf>
 13. \textls[0]Decoding the Diversity: A Review of the Indic AI Research Landscape - arXiv, accessed on April 3, 2026, <https://arxiv.org/html/2406.09559v1>
 14. Empirical study of pretrained multilingual language models for zero-shot cross-lingual knowledge transfer in generation - arXiv, accessed on April 3, 2026,
<https://arxiv.org/html/2310.09917v3>
 15. Enhancing Code-mixed Text Generation Using Synthetic Data Filtering in Neural Machine Translation - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2023.conll-1.15/>
 16. Conditioning LLMs to Generate Code-Switched Text: A Methodology Grounded in Naturally Occurring Data - arXiv, accessed on April 3, 2026,
<https://arxiv.org/html/2502.12924v1>
 17. Curriculum Learning for Cross-Lingual Data-to-Text Generation With Noisy Data - arXiv, accessed on April 3, 2026, <https://arxiv.org/html/2412.13484v1>
 18. Code-Switching Curriculum Learning for Multilingual Transfer in LLMs - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2025.findings-acl.407.pdf>
 19. IndicBART: A Pre-trained Model for Natural Language Generation of Indic Languages, accessed on April 3, 2026,
https://www.researchgate.net/publication/354435178_IndicBART_A_Pre-trained_

[Model_for_Natural_Language_Generation_of_Indic_Languages](#)

20. ilsum 2023, accessed on April 3, 2026, <https://ilsum.github.io/ilsum/2023/index.html>
21. Advances in Multilingual Machine Translation and Evaluation for Indian Languages, accessed on April 3, 2026, <https://ufal.mff.cuni.cz/mtm24/files/02-advances-in-multilingual-mt-raj-dabre.pdf>
22. Abstractive Text Summarization for Hindi Language using IndicBART - CEUR-WS.org, accessed on April 3, 2026, <https://ceur-ws.org/Vol-3395/T6-5.pdf>
23. ILSUM2024, accessed on April 3, 2026, <https://ilsum.github.io/>
24. accessed on April 3, 2026, [https://www.emergentmind.com/topics/hindi-text-summarization-dataset#:~:text=GupShup%20\(2021\)%3A%20A%20code,et%20al.%2C%202021\).](https://www.emergentmind.com/topics/hindi-text-summarization-dataset#:~:text=GupShup%20(2021)%3A%20A%20code,et%20al.%2C%202021).)
25. GupShup: An Annotated Corpus for Abstractive Summarization of Open-Domain Code-Switched Conversations - Semantic Scholar, accessed on April 3, 2026, <https://www.semanticscholar.org/paper/GupShup%3A-An-Annotated-Corpus-for-Abstractive-of-Mehnaz-Mahata/5563503767e65b32d8da7a050506fa0fb434e349>
26. [2104.08578] GupShup: An Annotated Corpus for Abstractive Summarization of Open-Domain Code-Switched Conversations - arXiv, accessed on April 3, 2026, <https://arxiv.org/abs/2104.08578>
27. GupShup: Summarizing Open-Domain Code-Switched Conversations - ACL Anthology, accessed on April 3, 2026, <https://aclanthology.org/2021.emnlp-main.499.pdf>
28. DravidianCodeMix: sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text - PMC, accessed on April 3, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9388449/>
29. HinGE: A Dataset for Generation and Evaluation of Code-Mixed Hinglish Text, accessed on April 3, 2026, <https://aclanthology.org/2021.eval4nlp-1.20/>
30. HinGE: A Dataset for Generation and Evaluation of Code-Mixed Hinglish Text - ACL Anthology, accessed on April 3, 2026, <https://aclanthology.org/2021.eval4nlp-1.20.pdf>
31. Language Modelling for Code-Switched Text - Saurabh Garg, accessed on April 3, 2026, <https://saurabhgarg1996.github.io/files/7.pdf>
32. yinxx/MUSE - Gitee, accessed on April 3, 2026, <https://gitee.com/yinxx/MUSE>
33. GitHub - facebookresearch/MUSE: A library for Multilingual Unsupervised or Supervised word Embeddings, accessed on April 3, 2026, <https://github.com/facebookresearch/MUSE>
34. PanLex | PanLex, accessed on April 3, 2026, <https://panlex.org/>
35. Part of Speech Tagging with Stop words using NLTK in python - GeeksforGeeks, accessed on April 3, 2026, <https://www.geeksforgeeks.org/python/part-speech-tagging-stop-words-using-nltk-python/>
36. Categorizing and POS Tagging with NLTK Python - Learntek, accessed on April 3, 2026, <https://www.learntek.org/blog/categorizing-pos-tagging-nltk-python/>
37. CodeMixBench: Evaluating Large Language Models on Code Generation with

- Code-Mixed Prompts - arXiv, accessed on April 3, 2026,
<https://arxiv.org/html/2505.05063v1>
38. CodeMixed Text Generator (GCM) - GitHub, accessed on April 3, 2026,
<https://github.com/microsoft/CodeMixed-Text-Generator>
 39. GCM: A Toolkit for Generating Synthetic Code-mixed Text - ACL ..., accessed on April 3, 2026, <https://aclanthology.org/2021.eacl-demos.24/>
 40. Sample-Efficient Language Model for Hinglish Conversational AI - arXiv, accessed on April 3, 2026, <https://arxiv.org/html/2504.19070v1>
 41. Indic NLP Library - Anoop Kunchukuttan, accessed on April 3, 2026,
https://anoopkunchukuttan.github.io/indic_nlp_library/
 42. normalize Package - Indic NLP Library 0.2 documentation - Read the Docs, accessed on April 3, 2026,
<https://indic-nlp-library.readthedocs.io/en/latest/indicnlp.normalize.html>
 43. Unicode Normalization and Grapheme Parsing of Indic Languages - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2024.lrec-main.1479.pdf>
 44. Script Gap: Evaluating LLM Triage on Indian Languages in Native vs Roman Scripts in a Real World Setting - arXiv, accessed on April 3, 2026,
<https://arxiv.org/html/2512.10780v1>
 45. Unified NMT models for the Indian subcontinent, transcending script-barriers - ACL Anthology, accessed on April 3, 2026,
<https://aclanthology.org/2022.deeplo-1.23.pdf>
 46. ai4bharat-transliteration - PyPI, accessed on April 3, 2026,
<https://pypi.org/project/ai4bharat-transliteration/>
 47. indic-transliteration/indic_transliteration_py: Python package for indic script transliteration - GitHub, accessed on April 3, 2026,
https://github.com/indic-transliteration/indic_transliteration_py
 48. IndicBART: A Pre-trained Model for Indic Natural Language Generation of Indic Languages - Microsoft Research, accessed on April 3, 2026,
<https://www.microsoft.com/en-us/research/publication/indicbart-a-pre-trained-model-for-indic-natural-language-generation-of-indic-languages/>
 49. Interpreting Hinglish Conversations | by Sayan Biswas | inspiringbrilliance - Medium, accessed on April 3, 2026,
<https://medium.com/inspiredbrilliance/interpreting-hinglish-conversations-79dab7cabd47>
 50. ai4bharat/MultiIndicSentenceSummarizationSS - Hugging Face, accessed on April 3, 2026, <https://huggingface.co/ai4bharat/MultiIndicSentenceSummarizationSS>
 51. Code-Mixed Hindi Queries - Emergent Mind, accessed on April 3, 2026,
<https://www.emergentmind.com/topics/code-mixed-hindi-queries>
 52. Sentiment analysis of code-mixed Dravidian languages leveraging pretrained model and word-level language tag - Cambridge University Press, accessed on April 3, 2026,
<https://www.cambridge.org/core/journals/natural-language-processing/article/sentiment-analysis-of-codemixed-dravidian-languages-leveraging-pretrained-model-and-wordlevel-language-tag/024F7BA5EE3F5F29CDB20EC9AD79CCE5>

53. Curriculum Learning Strategies for Hindi-English Codemixed Sentiment Analysis - arXiv.org, accessed on April 3, 2026, <https://arxiv.org/abs/1906.07382>
54. Enhancing Extractive Summarization for Low Resource Indian Languages using TF-IDF and SVD - CEUR-WS.org, accessed on April 3, 2026, <https://ceur-ws.org/Vol-4054/T1-5.pdf>
55. prajdabre/yanmtt: Yet Another Neural Machine Translation Toolkit - GitHub, accessed on April 3, 2026, <https://github.com/prajdabre/yanmtt>
56. ROUGE: Decoding the Quality of Machine-Generated Text - Analytics Vidhya, accessed on April 3, 2026, <https://www.analyticsvidhya.com/blog/2025/03/rouge/>
57. ROUGE-SS: A New ROUGE Variant for the Evaluation of Text Summarization | Request PDF, accessed on April 3, 2026, https://www.researchgate.net/publication/381266266_ROUGE-SS_A_New_ROUGE_Variant_for_the_Evaluation_of_Text_Summarization
58. CoMix: Guide Transformers to Code-Mix using POS structure and Phonetics - Semantic Scholar, accessed on April 3, 2026, <https://pdfs.semanticscholar.org/6e4a/a48948f62a5b9a12e492db36b02892f6484d.pdf>
59. Summarizing Indian Languages using Multilingual Transformers based Models - CEUR-WS.org, accessed on April 3, 2026, <https://ceur-ws.org/Vol-3395/T6-8.pdf>